

Chapitre 3 : Structures de données avancées

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Fil rouge** : le registre des colis de Yonnee Express · **Version** : 2026-10-23

Fiche du chapitre

Objectifs	A la fin du chapitre, l'apprenant sait : 1) choisir entre liste, ensemble, dictionnaire et tuple selon l'usage et le **coût** des opérations ; 2) utiliser les opérations d'ensembles ; 3) construire et parcourir des dictionnaires imbriqués ; 4) compter avec `Counter` et regrouper avec `defaultdict` ; 5) gérer une file avec `deque` ; 6) donner des noms aux champs d'un tuple avec `namedtuple` ; 7) trier et chercher le minimum ou le maximum selon un critère (`key`) ; 8) parcourir plusieurs séquences avec `zip` et `enumerate` .
Prérequis	Chapitres 1 et 2 ; listes, dictionnaires et ensembles de L1
Durée	🟢 Débutant : 7 h · 🔵 Standard : 5 h · 🟣 Expert : 3 h 30 (atelier compris)
Matériel	Python 3.12 ou plus récent (vérifié avec Python 3.13), VS Code ; le registre complet (fourni)
Livrable	L'atelier 3 : les statistiques des livreurs et des quartiers, publiées sur GitHub

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Créez un dossier `essais-ch3` . Commencez par y créer le fichier `donnees.py` ci-dessous : tous les exemples du chapitre l'**importent** (`from donnees import LIVRAISONS`), pour ne pas recopier les données à chaque fois. Le chapitre 6 expliquera en détail ce mécanisme d'import.

```
LIVRAISONS = [
    {"numero": 1, "livreur": "Awa", "quartier": "Médina", "poids": 2.5, "frais": 1000,
         "duree_min": 28},
    {"numero": 2, "livreur": "Modou", "quartier": "Pikine", "poids": 7.0, "frais": 3000,
         "duree_min": 65},
    {"numero": 3, "livreur": "Awa", "quartier": "Yoff", "poids": 1.2, "frais": 2000, "duree_min":
        41},
    {"numero": 4, "livreur": "Fatou", "quartier": "Médina", "poids": 0.8, "frais": 1000,
         "duree_min": 22},
    {"numero": 5, "livreur": "Modou", "quartier": "Plateau", "poids": 5.5, "frais": 2000,
         "duree_min": 47},
    {"numero": 6, "livreur": "Awa", "quartier": "Pikine", "poids": 3.0, "frais": 2500,
         "duree_min": 58},
    {"numero": 7, "livreur": "Fatou", "quartier": "Yoff", "poids": 9.1, "frais": 2500,
         "duree_min": 39},
]
```

1. Choisir la bonne structure

En L1, vous avez utilisé des listes et des dictionnaires. Chaque structure a ses forces, et surtout ses **coûts** : certaines opérations sont instantanées, d'autres prennent un temps proportionnel à la taille des données. Sur 7 livraisons, on ne voit pas la différence ; sur 500 000 (chapitre 5), elle devient énorme. Créez `essai1.py` :

```
import time

numeros = list(range(1_000_000))
ensemble = set(numeros)

debut = time.perf_counter()
for _ in range(100):
    999_999 in numeros
duree_liste = time.perf_counter() - debut

debut = time.perf_counter()
for _ in range(100):
    999_999 in ensemble
duree_ensemble = time.perf_counter() - debut

print(f"Liste    : {duree_liste * 1000:.1f} ms pour 100 recherches")
print(f"Ensemble : {duree_ensemble * 1000:.3f} ms pour 100 recherches")
print(f"L'ensemble est environ {duree_liste / duree_ensemble:,.0f} fois plus rapide".replace(",",
        " "))
```

```
Liste    : 1189.2 ms pour 100 recherches
Ensemble : 0.020 ms pour 100 recherches
L'ensemble est environ 60 987 fois plus rapide
```

(Vos durées seront différentes : elles dépendent de l'ordinateur. Le rapport reste énorme.)

Morceau	Ce qu'il veut dire
`1_000_000`	Un million ; les `_` ne servent qu'à rendre le nombre lisible
`time.perf_counter()`	Un chronomètre précis : la différence entre deux appels donne une durée en secondes
`999_999 in numeros`	Pour une **liste**, Python compare 999 999 à chaque élément, un par un, depuis le début : un million de comparaisons
`999_999 in ensemble`	Pour un **ensemble**, Python calcule directement où devrait se trouver l'élément (on dit qu'il le « hache ») : une seule vérification, quelle que soit la taille

Opération	Liste	Ensemble (`set`)	Dictionnaire (`dict`)
`x in ...`	**Lent** : proportionnel à la taille	**Instantané**	**Instantané** (sur les clés)
Ajouter à la fin	Instantané (`append`)	Instantané (`add`)	Instantané (`d[cle] = ...`)
Ajouter au début	**Lent** (`insert(0, x)` décale tout)	(pas d'ordre)	(pas de « début »)
Accès par position	Instantané (`liste[3]`)	Impossible	Impossible (on accède par clé)
Doublons	Autorisés	**Impossibles**	Clés uniques
Ordre	Conservé	**Aucun ordre garanti**	Ordre d'insertion conservé

> 🔵 **Standard : la notation O** Les informaticiens notent ces coûts **O(n)** (« proportionnel à n », le nombre d'éléments) et **O(1)** (« constant », indépendant de n). Chercher dans une liste est O(n) ; dans un ensemble ou un dictionnaire, O(1) en moyenne. Retenez la règle pratique : **si vous testez souvent `x in ...`, utilisez un ensemble ou un dictionnaire**.

2. Les ensembles et leurs opérations

Un **ensemble** (`set`) contient des éléments **sans doublon** et sans ordre. Il est parfait pour comparer des groupes. Créez `essai2.py` :

```
from donnees import LIVRAISONS

quartiers_awa = {l["quartier"] for l in LIVRAISONS if l["livreur"] == "Awa"}
quartiers_modou = {l["quartier"] for l in LIVRAISONS if l["livreur"] == "Modou"}
print("Awa :", sorted(quartiers_awa))
print("Modou :", sorted(quartiers_modou))
print("Les deux :", quartiers_awa & quartiers_modou)
print("Au moins un des deux :", sorted(quartiers_awa | quartiers_modou))
print("Awa seulement :", sorted(quartiers_awa - quartiers_modou))
```

```
Awa : ['Médina', 'Pikine', 'Yoff']
Modou : ['Pikine', 'Plateau']
Les deux : {'Pikine'}
Au moins un des deux : ['Médina', 'Pikine', 'Plateau', 'Yoff']
Awa seulement : ['Médina', 'Yoff']
```

Morceau	Ce qu'il veut dire
`{l["quartier"] for l in ... if ...}`	Un ensemble **en compréhension** (accolades) : les quartiers d'Awa, chacun **une seule fois**
`sorted(ensemble)`	Une **liste** triée des éléments (un ensemble n'a pas d'ordre : on trie pour afficher)
`a & b`	L'**intersection** : les éléments présents dans les deux
`a | b`	L'**union** : les éléments présents dans au moins un
`a - b`	La **différence** : dans `a` mais pas dans `b`
`a ^ b`	La **différence symétrique** : dans un seul des deux

> 🧨 **Essayez** : écrivez `quartiers_awa[0]`. Message : `TypeError: 'set' object is not subscriptable` (« un ensemble ne s'indexe pas ») : il n'y a pas de « premier » élément dans un ensemble. Annulez.

3. Les dictionnaires imbriqués

Un dictionnaire peut contenir d'autres dictionnaires : c'est la façon naturelle de ranger des informations **par catégorie**. Construisons une fiche par livreur, avec le nombre de livraisons, le total des frais, et le nombre de livraisons par quartier. Créez `essai3.py` :

```python
from donnees import LIVRAISONS

par_livreur = {}
for l in LIVRAISONS:
    fiche = par_livreur.setdefault(l["livreur"], {"livraisons": 0, "frais": 0, "quartiers": {}})
    fiche["livraisons"] += 1
    fiche["frais"] += l["frais"]
    fiche["quartiers"][l["quartier"]] = fiche["quartiers"].get(l["quartier"], 0) + 1

print(par_livreur["Awa"])
print(par_livreur["Awa"]["quartiers"]["Pikine"])
print(par_livreur.get("Ibou", {}).get("frais", 0))
```

```
{'livraisons': 3, 'frais': 5500, 'quartiers': {'Médina': 1, 'Yoff': 1, 'Pikine': 1}}
1
0
```

Morceau	Ce qu'il veut dire
`par_livreur.setdefault(cle, valeur)`	Renvoie la valeur de la clé ; si la clé n'existe pas, la crée d'abord avec `valeur`
`fiche = ...`	`fiche` **est** le dictionnaire rangé dans `par_livreur` (pas une copie) : le modifier modifie `par_livreur`
`fiche["quartiers"][l["quartier"]]`	On descend d'un niveau : le dictionnaire des quartiers de ce livreur, puis le quartier
`par_livreur["Awa"]["quartiers"]["Pikine"]`	Trois niveaux : le livreur, la rubrique, le quartier
`.get("Ibou", {}).get("frais", 0)`	Accès **sans risque** : si Ibou n'existe pas, on obtient un dictionnaire vide, puis 0

> 🪛 **Essayez** : écrivez `print(par_livreur["Ibou"]["frais"])`. Message : `KeyError: 'Ibou'`. Dans une structure imbriquée, chaque niveau peut manquer : `get` avec une valeur par défaut évite le plantage. Annulez.

4. Compter avec `Counter`

Compter est la première opération de toute statistique. Le module `collections` (fourni avec Python) propose `Counter`, un dictionnaire spécialisé. Créez `essai4.py` :

```
from collections import Counter

from donnees import LIVRAISONS

quartiers = Counter(l["quartier"] for l in LIVRAISONS)
print(quartiers)
print(quartiers.most_common(2))
print(quartiers["Médina"], quartiers["Ouakam"])
quartiers.update(["Yoff", "Yoff", "Ouakam"])
print(quartiers.most_common(1), quartiers["Ouakam"])
```

```
Counter({'Médina': 2, 'Pikine': 2, 'Yoff': 2, 'Plateau': 1})
[('Médina', 2), ('Pikine', 2)]
2 0
[('Yoff', 4)] 1
```

Morceau	Ce qu'il veut dire
`Counter(l["quartier"] for l in LIVRAISONS)`	Compte combien de fois chaque quartier apparaît
`.most_common(2)`	Les 2 plus fréquents, sous forme de liste de tuples `(élément, nombre)`

Morceau	Ce qu'il veut dire
`quartiers["Ouakam"]`	**0** pour un élément absent, sans `KeyError` (contrairement à un dictionnaire)
`.update([...])`	Ajoute de nouvelles occurrences au comptage

Comparez avec ce qu'il fallait écrire au chapitre 2 : `compteur[cle] = compteur.get(cle, 0) + 1` dans une boucle. `Counter` fait la même chose en une ligne.

> 🟢 **Débutant : `most_common` et les égalités** `most_common(2)` a renvoyé Médina et Pikine, alors que Yoff a **aussi** 2 livraisons. En cas d'égalité, `most_common` garde l'ordre d'apparition, sans le signaler. Pour savoir s'il y a des ex aequo, regardez les nombres (exercice 1).

5. Regrouper avec `defaultdict`

Pour **regrouper** des valeurs par catégorie (les durées de chaque livreur), on veut une liste par clé, créée automatiquement la première fois. C'est le rôle de `defaultdict`. Créez `essai5.py` :

```
from collections import defaultdict

from donnees import LIVRAISONS

durees = defaultdict(list)
for l in LIVRAISONS:
    durees[l["livreur"]].append(l["duree_min"])
print(dict(durees))
for livreur, liste in durees.items():
    print(f"{livreur:6} {sum(liste) / len(liste):5.1f} min en moyenne")
print(durees["Ibou"])
```

```
{'Awa': [28, 41, 58], 'Modou': [65, 47], 'Fatou': [22, 39]}
Awa     42.3 min en moyenne
Modou   56.0 min en moyenne
Fatou   30.5 min en moyenne
[]
```

Morceau	Ce qu'il veut dire
`defaultdict(list)`	Un dictionnaire qui, pour une clé absente, crée **automatiquement** une valeur avec `list()`, c'est-à-dire une liste vide
`durees[livreur].append(...)`	Fonctionne même la première fois : plus besoin de `setdefault`
`defaultdict(int)`	La même chose avec `int()`, qui vaut 0 : pratique pour additionner (`totaux[cle] += montant`)
`dict(durees)`	Un dictionnaire normal, pour l'affichage

> 🖍️ **Essayez** : après `print(durees["Ibou"])`, affichez `list(durees)`. Ibou fait maintenant partie des clés ! Lire une clé absente d'un `defaultdict` la **crée**. Pour tester sans créer, utilisez `"Ibou" in durees`.

6. Une file d'attente avec `deque`

Les colis arrivent au dépôt et partent **dans l'ordre d'arrivée** : premier arrivé, premier livré. C'est une **file** (en anglais *queue*). Avec une liste, retirer le premier élément (`liste.pop(0)`) oblige Python à décaler tous les autres : c'est lent sur de grandes listes. `deque` (prononcez « dèque », *double-ended queue*) ajoute et retire aux deux bouts instantanément. Créez `essai6.py` :

```
from collections import deque

file = deque(["colis 101", "colis 102"])
file.append("colis 103")
premier = file.popleft()
print("On livre :", premier)
print("Il reste :", list(file))

derniers = deque(maxlen=3)
for numero in [201, 202, 203, 204, 205]:
    derniers.append(numero)
print("Les 3 derniers colis scannés :", list(derniers))
```

```
On livre : colis 101
Il reste : ['colis 102', 'colis 103']
Les 3 derniers colis scannés : [203, 204, 205]
```

Morceau	Ce qu'il veut dire
`append(x)` / `appendleft(x)`	Ajoute à droite (fin) / à gauche (début)
`pop()` / `popleft()`	Retire et renvoie l'élément de droite / de gauche
`deque(maxlen=3)`	Une file de **taille limitée** : quand elle est pleine, ajouter à droite fait sortir l'élément de gauche. Parfait pour garder « les N derniers »

7. Des tuples avec des noms : `namedtuple`

Un dictionnaire par livraison, c'est pratique, mais on peut se tromper de clé (`l["quartie"]`) sans que Python le voie avant l'exécution. Un **tuple nommé** donne un **nom** à chaque position d'un tuple. Créez `essai7.py` :

```python
from collections import namedtuple

Livraison = namedtuple("Livraison", ["numero", "livreur", "quartier", "frais"])
l = Livraison(8, "Ibou", "Parcelles", 2000)
print(l)
print(l.quartier, l[2])
numero, livreur, quartier, frais = l
print(livreur, frais)
print(l._replace(frais=2500))
print(l._asdict())
try:
    l.frais = 3000
except AttributeError as e:
    print("Erreur :", e)
```

```
Livraison(numero=8, livreur='Ibou', quartier='Parcelles', frais=2000)
Parcelles Parcelles
Ibou 2000
Livraison(numero=8, livreur='Ibou', quartier='Parcelles', frais=2500)
{'numero': 8, 'livreur': 'Ibou', 'quartier': 'Parcelles', 'frais': 2000}
Erreur : can't set attribute
```

Morceau	Ce qu'il veut dire
`namedtuple("Livraison", [...])`	Crée un **nouveau type** de tuple, nommé `Livraison`, avec quatre champs
`Livraison(8, "Ibou", ...)`	Une livraison, valeurs dans l'ordre des champs
`l.quartier`	Accès par **nom** (plus lisible que `l[2]`, qui marche aussi)
`numero, livreur, quartier, frais = l`	Le **déballage** : chaque variable reçoit une valeur, dans l'ordre
`l._replace(frais=2500)`	Une **nouvelle** livraison avec un champ changé ; `l` n'est pas modifiée
`l._asdict()`	La livraison sous forme de dictionnaire (pour l'écrire en JSON, par exemple)
`can't set attribute`	Comme tout tuple, un tuple nommé est **non modifiable**

> 🟢 **Débutant : alors, dictionnaire ou tuple nommé ?** Un **tuple nommé** quand tous les enregistrements ont **les mêmes champs** et ne changent pas (une livraison enregistrée). Un **dictionnaire** quand les champs varient ou qu'il faut les modifier. Au chapitre 7, vous découvrirez une troisième solution, plus puissante : la **classe**.

8. Trier et chercher selon un critère

`sorted` et `max` comparent normalement les éléments eux-mêmes. Avec `key`, on leur dit **selon quoi** comparer. Créez `essai8.py` :

```python
from donnees import LIVRAISONS

par_duree = sorted(LIVRAISONS, key=lambda l: l["duree_min"], reverse=True)
print([(l["numero"], l["duree_min"]) for l in par_duree[:3]])

par_livreur_puis_frais = sorted(LIVRAISONS, key=lambda l: (l["livreur"], -l["frais"]))
print([(l["livreur"], l["frais"]) for l in par_livreur_puis_frais])

plus_lourd = max(LIVRAISONS, key=lambda l: l["poids"])
print("Le plus lourd :", plus_lourd["numero"], plus_lourd["poids"], "kg")
```

```
[(2, 65), (6, 58), (5, 47)]
[('Awa', 2500), ('Awa', 2000), ('Awa', 1000), ('Fatou', 2500), ('Fatou', 1000), ('Modou', 3000),
('Modou', 2000)]
Le plus lourd : 7 9.1 kg
```

Morceau	Ce qu'il veut dire
`key=lambda l: l["duree_min"]`	Pour comparer deux livraisons, on compare leur durée. `lambda l: ...` est une petite fonction sans nom (chapitre 4) qui reçoit une livraison et renvoie la valeur à comparer
`reverse=True`	Du plus grand au plus petit
`key=lambda l: (l["livreur"], -l["frais"])`	Un **tuple** comme clé : on trie d'abord par livreur ; à livreur égal, par frais. Le signe `-` inverse l'ordre des frais : du plus cher au moins cher
`par_duree[:3]`	Les 3 premiers éléments de la liste triée
`max(..., key=...)`	L'élément dont la clé est la plus grande (`min` pour la plus petite)

`sorted` renvoie une **nouvelle** liste ; `liste.sort(key=...)` trie la liste **elle-même**, sans rien renvoyer.

> 🖍️ **Essayez** : écrivez `sorted(LIVRAISONS)` sans `key`. Message : `TypeError: '<' not supported between instances of 'dict' and 'dict'` : Python ne sait pas dire si un dictionnaire est « plus petit » qu'un autre. Annulez.

9. Parcourir à plusieurs : `zip` et `enumerate`

Créez `essai9.py` :

```python
quartiers = ["Médina", "Plateau", "Yoff", "Parcelles", "Pikine"]
tarifs = [1000, 1500, 2000, 2000, 2500]

for quartier, tarif in zip(quartiers, tarifs):
    print(f"{quartier:10} {tarif} F")
print(dict(zip(quartiers, tarifs)))

for rang, quartier in enumerate(quartiers, start=1):
    print(rang, quartier)
print(list(zip(quartiers, [1, 2])))
```

```
Médina     1000 F
Plateau    1500 F
Yoff       2000 F
Parcelles  2000 F
Pikine     2500 F
{'Médina': 1000, 'Plateau': 1500, 'Yoff': 2000, 'Parcelles': 2000, 'Pikine': 2500}
1 Médina
2 Plateau
3 Yoff
4 Parcelles
5 Pikine
[('Médina', 1), ('Plateau', 2)]
```

Morceau	Ce qu'il veut dire
`zip(a, b)`	Parcourt `a` et `b` **en même temps** : à chaque tour, un élément de chacun
`dict(zip(cles, valeurs))`	Fabrique un dictionnaire à partir de deux listes
`enumerate(liste, start=1)`	Donne le **numéro** de chaque élément en plus de l'élément
`zip(quartiers, [1, 2])`	`zip` s'arrête à la **plus courte** des deux : 2 tuples seulement, **sans prévenir**

> 🔵 **Standard : `zip(..., strict=True)`** Depuis Python 3.10, `zip(a, b, strict=True)` lève une `ValueError` si les deux séquences n'ont pas la même longueur, au lieu de s'arrêter en silence. Utilisez-le quand les deux listes **doivent** correspondre (une liste de quartiers et leur liste de tarifs).

10. Combiner : le taux de retard par livreur

Les outils se combinent. Avec deux `Counter`, calculons la part des livraisons de plus de 45 minutes de chaque livreur. Créez `essai10.py` :

```python
from collections import Counter

from donnees import LIVRAISONS

total = Counter(l["livreur"] for l in LIVRAISONS)
retards = Counter(l["livreur"] for l in LIVRAISONS if l["duree_min"] > 45)
print(retards)
for livreur in sorted(total):
    print(f"{livreur:6} {retards[livreur]} sur {total[livreur]} : {retards[livreur] /
         total[livreur]:.0%} de retards")
```

```
Counter({'Modou': 2, 'Awa': 1})
Awa    1 sur 3 : 33% de retards
Fatou  0 sur 2 : 0% de retards
Modou  2 sur 2 : 100% de retards
```

Fatou n'a aucun retard : `retards["Fatou"]` vaut 0 grâce à `Counter`, sans `KeyError`. Mais attention aux conclusions : 2 livraisons ne suffisent pas à juger un livreur. C'est ce que vous ferez sérieusement, sur 2 000 livraisons, dans l'atelier.

Exercices

Créez un dossier `exercices-ch3`.

- ★ **Exercice 1 : l'élection du délégué.** Les votes : Aminata, Cheikh, Aminata, Pape, Cheikh, Aminata, Pape, Cheikh, Cheikh, Aminata, Pape. Comptez les voix avec `Counter`, et affichez « élu : … » ou, en cas d'égalité, « égalité entre … et … ». Ajoutez ensuite un vote pour Pape : que renvoie `most_common(1)` ? Pourquoi est-ce trompeur ? (Section 4.)
- ★ **Exercice 2 : les inscriptions.** En algorithmique : Awa, Moussa, Khady, Ibrahima, Sokhna ; en bases de données : Moussa, Sokhna, Ousmane, Fatou. Qui suit les deux cours ? Au moins un ? Seulement l'algorithmique ? Un seul des deux ? (Section 2.)
- ★★ **Exercice 3 : les anagrammes.** Regroupez les mots `chien, marie, niche, aimer, sel, chine, les, rame, ramie, mare` par familles d'anagrammes (les mêmes lettres dans un autre ordre), et n'affichez que les familles d'au moins deux mots. Indice : deux anagrammes ont la même « signature » `"".join(sorted(mot))`, à utiliser comme clé d'un `defaultdict(list)`. Attendu : 4 familles. (Section 5.)
- ★★ **Exercice 4 : l'historique du navigateur.** Un navigateur garde les 4 dernières pages visitées. Visitez dans l'ordre : accueil, cours, chapitre-1, chapitre-2, exercices, corrigés. Affichez l'historique, puis simulez le bouton « Retour » : la page actuelle est retirée, et on revient à la précédente. Attendu : `['chapitre-1', 'chapitre-2', 'exercices', 'corrigés']`, puis retour sur `exercices`. (Section 6.)
- ★★★ **Exercice 5 : le classement avec ex aequo.** Avec un `namedtuple` `Eleve(nom, moyenne)` : Diallo 14,5 ; Ba 16 ; Sarr 14,5 ; Ndiaye 12,25 ; Gueye 16. Affichez le classement par moyenne décroissante, puis par nom à moyenne égale, avec le **rang** à la manière des concours : deux ex aequo ont le même rang, et le suivant « saute » (1, 1, 3, 3, 5). (Sections 7 et 8.)

Atelier 3 : les statistiques du registre

Le travail principal du chapitre : sur le registre complet de Yonnee Express (2 000 livraisons, de janvier 2025 à octobre 2026), vous calculez les statistiques que la direction demande : livraisons par quartier et par livreur, durées, tableau croisé, taux de retard, frais par mois, dernières livraisons de chaque livreur. Chaque question se résout avec l'outil le mieux adapté du chapitre. L'énoncé et le registre vous sont fournis séparément.

A retenir

- Tester `x in ...` : **ensemble** ou **dictionnaire** (instantané), pas une liste (lent sur de grandes données).
- Ensembles : `&` (les deux), `|` (au moins un), `-` (l'un sans l'autre), `^` (un seul) ; pas d'ordre, pas d'indice.
- Dictionnaires imbriqués : `setdefault` pour créer un niveau, `get(cle, defaut)` pour lire sans risque.
- `Counter` : compter (`most_common` , 0 pour un absent) ; `defaultdict(list)` : regrouper ; `defaultdict(int)` : additionner.
- `deque` : file (`append` , `popleft`) ; `maxlen` pour garder les N derniers.
- `namedtuple` : des champs nommés, non modifiables ; `_replace` , `_asdict` .
- `sorted(..., key=lambda x: ..., reverse=True)` ; une clé tuple pour plusieurs critères ; `min` / `max` avec `key` .
- `zip` s'arrête à la plus courte (`strict=True` pour vérifier) ; `enumerate(..., start=1)` numérote.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`TypeError: 'set' object is not subscriptable`	Accès par indice à un ensemble	`sorted(ensemble)[0]` , ou une liste
`TypeError: unhashable type: 'list'`	Une liste comme élément d'ensemble ou clé de dictionnaire	Utiliser un tuple
`KeyError: '...'` dans un dictionnaire imbriqué	Un niveau manque	`.get(cle, {})` , ou `defaultdict`
Une clé apparaît « toute seule » dans un `defaultdict`	Une simple lecture d'une clé absente la crée	Tester avec `cle in d`
`AttributeError: can't set attribute`	Modification d'un champ de tuple nommé	`l = l._replace(champ=valeur)`
`TypeError: '<' not supported between instances of 'dict' and 'dict'`	Tri sans `key` d'une liste de dictionnaires	`sorted(..., key=lambda d: d["champ"])`

Message ou symptôme	Ce que ça veut dire	Solution
Le tri est dans le mauvais ordre	`reverse` oublié, ou clé numérique stockée en texte (`"10" < "9"`)	`reverse=True` ; convertir en nombre
`zip` donne moins de résultats que prévu	Les séquences n'ont pas la même longueur	`zip(..., strict=True)`
Programme très lent sur de grandes données	`x in liste` dans une boucle	Convertir la liste en ensemble une fois

Pour aller plus loin

- Le tutoriel officiel, « Structures de données » (en français) : https://docs.python.org/fr/3/tutorial/datastructures.html
- Le module `collections` : https://docs.python.org/fr/3/library/collections.html
- Le guide du tri en Python : https://docs.python.org/fr/3/howto/sorting.html
- Les coûts des opérations sur les structures de Python (en anglais) : https://wiki.python.org/moin/TimeComplexity